

ADR 0055 — Working-tree status readings carry a server-stamped `scanned_at`, additive to the v1 wire contract

Date: 2026-08-12 Status: Accepted — implemented

Supersedes nothing.

Context

2026-08-12, 00:58. Git-Vista displayed, for `mcp-fleet`, a branch and dirty-file badge from roughly 19 hours earlier — `main/8881bab` on disk, `fix/codex- adversarial-review-2026-08-11/ f59d52d` on screen — with no indication anywhere that the reading was not current. The operator acted on it: told another session there were uncommitted files that had in fact been committed hours earlier.

The topbar status chip (`GET /api/status`, `RepoStatus`) is fetched once when its Leptos resource's key changes (`(activity panel open, graph epoch)`) and then held in memory until the next fetch. Nothing refreshes it on a timer or on window focus. The server itself is never stale — `worktree_status` in `handlers/read.rs` shells out to `git status --porcelain=v2 --branch` fresh on every request — the staleness lives entirely on the client, in how long a fetched value sits unrefreshed before the operator looks at it again.

A stale reading and a fresh one were **pixel-identical**: same chip, same colour, same counts. That is the specific failure mode the sibling project `workboard` was built to avoid, and its

`refresh_verdict` names it exactly: *"A board that has stopped refreshing looks EXACTLY like a board where nothing is happening: same pages, same green chips, no error anywhere."* `RepoStatus` had no field to tell the two apart even if the client wanted to.

Decision

Add `scanned_at: i64` (Unix seconds) to `RepoStatus`, stamped by the server at the point it collects the reading, and render the reading's age in the topbar chip unconditionally — not behind a hover.

- `crates/git-vista-core/src/status.rs`: `RepoStatus` gains `#[serde(default)] pub scanned_at: i64`. The parser (`parse_porcelain_v2`) never sets it — it has no wall-clock access and shouldn't need one; `0` is its default and means "not yet stamped."
- `crates/git-vista-server/src/handlers/read.rs`: `worktree_status` sets `parsed.scanned_at = crate::activity::now_secs()` immediately after parsing — the instant closest to when `git status` actually ran, reusing the existing Unix-seconds helper already used for activity-feed timestamps.
- `crates/git-vista/src/datetime.rs`: two new pure, host-tested functions reusing the existing `ago_label` relative-time vocabulary (already proven in the Activity feed, `activity.rs:408`) rather than inventing a second one:
 - `freshness_label(delta_secs: Option<i64>) -> String` — "as of just now" / "as of 3h ago" / "as of over a week ago" / "age unknown" for `None`.
 - `is_stale(delta_secs: Option<i64>) -> bool` — `true` past `STALE_THRESHOLD_SECS` (5 minutes) **or when the age is unknown at all** — an undated reading gets no benefit of the doubt.

- `crates/git-vista/src/app/mod.rs`: the topbar chip computes `age = (scanned_at > 0).then(|| now - scanned_at)` (client now via `js_sys::Date::now()`), always shows a " · as of ..." suffix inline in the chip text (not only in the hover title), and appends a `stale` CSS class when `is_stale(age)` is true.
- `crates/git-vista/styles.css`: `.status-chip.stale` (dashed border, reduced opacity) stacks on top of whichever severity colour (clean/dirty/conflict) the chip already carries, rather than replacing it — staleness is a trust signal about the reading, not a fourth severity level competing with conflict/dirty/clean.

No `PROTOCOL_VERSION` bump. `scanned_at` is a genuinely additive field: both `RepoStatus` and the wire format have no `#[serde(deny_unknown_fields)]`, so an old client reading a new server's JSON ignores the extra key, and a new client reading an old server's JSON (missing the key) gets `0` via `#[serde(default)]` — which the client already treats as "age unknown," the correct reading of "an older peer never told me." Per `version.rs`'s own stated policy, a bump is for a shape change an older peer would *misread*; this isn't one.

Alternatives considered

Refresh automatically (poll or refetch-on-focus) instead of dating the reading. Rejected as the primary fix, though nothing here prevents adding it later. A refresh that fails silently leaves a stale reading looking fresh again — the same bug with a shorter fuse, and one that reappears exactly when the network or server is unhealthy, which is when an accurate status matters most. Making staleness *rarer* is not the same fix as making it *visible*; the handoff document that scoped this work called this out explicitly and asked for (1) first.

Attach the age only to the hover title, not the visible chip text. Rejected. The whole bug was that nothing was visible *without asking* — a title attribute is exactly "asking" (hover or long-press). The acceptance criterion (`age_is_visible`) requires the UI to state the age unprompted, so both landed: the visible chip text and the title both carry the freshness label now.

Migrate the chip to the v2 `WorktreeStatus` / `generation` token instead of touching v1 `RepoStatus`. Rejected for this change. `generation` is a content digest (ADR 0001) that detects *drift between two reads*, not *age of one read* — it answers "has this changed since I last looked," not "how long ago did I look." The chip's grouping logic (`chip_label`) also deliberately still reads v1 `RepoStatus` because `dialogs/commit.rs` needs its per-entry `ChangeKind` detail that v2 doesn't carry (documented in `features/status/core.rs`'s own module doc). Migrating the whole chip to v2 to gain an age field would be a much larger, unrelated change bundled into a bug fix.

A single combined boolean (`is_stale`) with no textual age. Rejected. A binary flag tells the operator "old" but not *how* old, and the live incident specifically needed "19 hours" to register as obviously wrong — "stale" alone might have read as a minor delay worth ignoring. The text carries the information that makes the staleness actionable.

Consequences

- `RepoStatus` gains one field; every construction site (production and test fixtures) needed the field added — one test fixture in `crates/git-vista/src/features/dialogs/commit.rs` was updated to `scanned_at: 0`.
- The topbar chip is measurably busier (an extra " · as of ..." clause) — accepted as the direct cost of the fix; a quieter chip that lies is worse than a busier one that doesn't.

- `scanned_at == 0` is now an overloaded-but-documented sentinel for "no server timestamp available," distinguished from a real Unix-epoch-zero reading (which cannot occur in practice — no git repository predates 1970). A future reader who instruments a raw `RepoStatus` outside the client should know `0` means "unknown," not "January 1970."
- **This does not fix the underlying "nothing refetches automatically" gap.** A tab left open for 19 hours will still show a stale, dashed-border chip rather than a fresh one — it will just say so honestly now, which was the actual bug. Automatic refresh remains a legitimate follow-up (see Alternatives), not obsoleted by this.
- Any future consumer of `RepoStatus` (MCP tools, other clients) inherits the `scanned_at` field for free and can apply the same `is_stale`/`freshness_label` reasoning without re-deriving it, since the logic lives in a plain, dependency- free module (`datetime.rs`) rather than inline in the Leptos view.

Where this is implemented

Concern	Location
<code>RepoStatus.scanned_at</code> field	<code>crates/git-vista-core/src/status.rs:69-97</code>
Server-side stamping	<code>crates/git-vista-server/src/handlers/read.rs</code> (<code>worktree_status</code> , after <code>parse_porcelain_v2</code>)
Reused Unix-seconds helper	<code>crates/git-vista-server/src/activity.rs:53-58</code> (<code>now_secs</code>)
Freshness label / staleness threshold	<code>crates/git-vista/src/datetime.rs</code> (<code>freshness_label</code> , <code>is_stale</code> , <code>STALE_THRESHOLD_SECS</code>)

Concern	Location
Existing relative-time vocabulary reused	<code>crates/git-vista/src/datetime.rs</code> <code>(ago_label)</code> , precedent in <code>crates/git-vista/src/activity.rs:408</code>
Topbar chip render	<code>crates/git-vista/src/app/mod.rs</code> (the <code>status.get().flatten().map(...)</code> block)
Stale styling	<code>crates/git-vista/styles.css</code> (<code>.status-chip.stale</code> , <code>.status-age</code>)
Backward-compatible deserialization	<code>crates/git-vista-core/src/status.rs</code> tests: <code>missing_scanned_at_deserializes_to_zero</code> , <code>scanned_at_round_trips_through_json</code>

SECURITY_MODEL.md annotation

None. This is a read-path UX fix — it adds a wall-clock timestamp to an existing read-only status response and changes no authorization, request shape an older peer could misread, or sandbox behaviour. `GET /api/status` was already session-authenticated and `no-store`; both are unchanged.

Signed: 2025 · 2026-08-12T01:26:29-04:00